

```
# Ask the user for a move:
fromTower, toTower = getPlayerMove(towers)

# Move the top disk from fromTower to toTower:
disk = towers[fromTower].pop()
towers[toTower].append(disk)
```

In a single turn, the player views the current state of the towers and enters a move. The program then updates the `towers` data structure. We've hid the details of these tasks in the `displayTowers()` and `getPlayerMove()` functions. These descriptive function names allow the `main()` function to provide a general overview of what the program does.

The next lines check whether the player has solved the puzzle by comparing the complete tower in `SOLVED_TOWER` to `towers["B"]` and `towers["C"]`:

```
# Check if the user has solved the puzzle:
if SOLVED_TOWER in (towers["B"], towers["C"]):
    displayTowers(towers) # Display the towers one last time.
    print("You have solved the puzzle! Well done!")
    sys.exit()
```

We don't compare it to `towers["A"]`, because that pole begins with an already complete tower; a player needs to form the tower on the B or C poles to solve the puzzle. Note that we reuse `SOLVED_TOWER` to make the starting towers and check whether the player solved the puzzle. Because `SOLVED_TOWER` is a constant, we can trust that it will always have the value we assigned to it at the beginning of the source code.

The condition we use is equivalent to but shorter than `SOLVED_TOWER == towers["B"]` or `SOLVED_TOWER == towers["C"]`, a Python idiom we covered in Chapter 6. If this condition is `True`, the player has solved the puzzle, and we end the program. Otherwise, we loop back for another turn.

The `getPlayerMove()` function asks the player for a disk move and validates the move against the game rules:

```
def getPlayerMove(towers):
    """Asks the player for a move. Returns (fromTower, toTower)."""
    while True: # Keep asking player until they enter a valid move.
```
